

Algoritmos em Grafos

Aula 09 – Busca em grafos

Ciência da Computação



Prof. Dr. Robson Medrado de Oliveira

robson@pucgoias.edu.br

Roteiro da aula

1. Motivação: por que buscar em grafos?
2. Construção de uma árvore de busca.
3. Algoritmo genérico de exploração.
4. Execução passo a passo em um grafo desconexo.
5. Busca em profundidade (DFS).
6. Detecção de ciclos e consequências.

Objetivo da aula

Compreender como uma busca explora sistematicamente um grafo e como a árvore gerada registra os vértices alcançados a partir de um vértice inicial.

Ideia central

A busca transforma a exploração de um grafo em um processo organizado: marcar vértices, registrar predecessores e construir uma árvore de busca.



Motivação

Por que estudar busca em grafos?

Algoritmos de busca dão informações sobre a estrutura de um grafo, especialmente sobre **conexão** e **caminhos**. A mesma ideia básica aparece em vários problemas computacionais.

Pergunta central

Dado um grafo G e um vértice inicial s , quais vértices podem ser alcançados a partir de s ?

Aplicações

- ▶ componentes conexas;
- ▶ detecção de ciclos;
- ▶ distância entre vértices;
- ▶ bipartição;
- ▶ arestas de corte;
- ▶ labirintos e redes.

Árvores e grafos

Em árvores

Há formas naturais de exploração:

- ▶ pré-ordem;
- ▶ pós-ordem;
- ▶ percurso por níveis.

Em grafos

Em grafos gerais, ciclos e múltiplos caminhos tornam a exploração menos direta. Portanto, precisamos definir uma **ordem sistemática** de visita.

Busca em grafos = exploração organizada de vértices e arestas.



Árvore de busca

Ideia dos algoritmos de busca

A ideia central é construir, passo a passo, uma árvore T dentro de G , a partir de um vértice inicial s .

Expansão da árvore

Enquanto existir uma aresta $xy \in E(G)$ tal que

$$x \in V(T) \quad \text{e} \quad y \in V(G) \setminus V(T),$$

fazemos:

$$V(T) \leftarrow V(T) \cup \{y\}, \quad E(T) \leftarrow E(T) \cup \{xy\}.$$

Critério de parada

$$V(T) = V(G) \Rightarrow G \text{ é conexo}, \quad V(T) \neq V(G) \Rightarrow G \text{ é desconexo}.$$

Algoritmo genérico de busca

Busca(G, s)

- 1 $V(T) \leftarrow \{s\}$ e $E(T) \leftarrow \emptyset$
- 2 enquanto existir $xy \in E(G)$ com $x \in V(T)$ e $y \notin V(T)$ faça
- 3 $V(T) \leftarrow V(T) \cup \{y\}$
- 4 $E(T) \leftarrow E(T) \cup \{xy\}$
- 5 retorne T

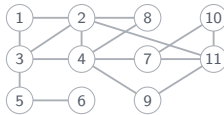
Observação

A árvore de busca pode não ser única: ela depende da escolha da aresta xy em cada iteração.



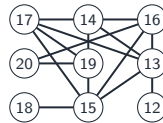
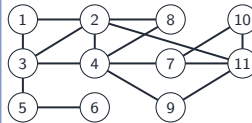
Execução passo a passo

Execução da busca



$Busca(G, 7)$

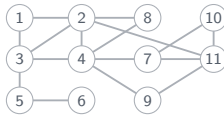
G



$V(T)$

$V(G) \setminus V(T)$

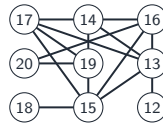
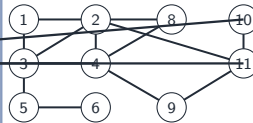
Execução da busca



Busca($G, 7$)

G

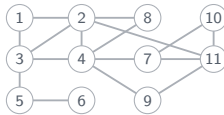
7



$V(T)$

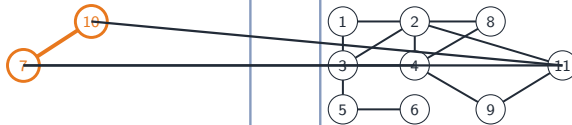
$V(G) \setminus V(T)$

Execução da busca



Busca($G, 7$)

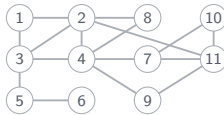
G



$V(T)$

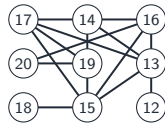
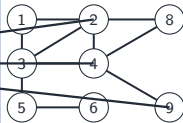
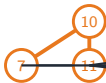
$V(G) \setminus V(T)$

Execução da busca



Busca($G, 7$)

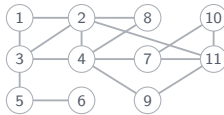
G



$V(T)$

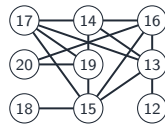
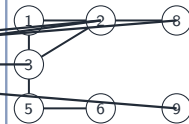
$V(G) \setminus V(T)$

Execução da busca



Busca($G, 7$)

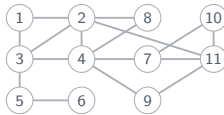
G



$V(T)$

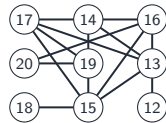
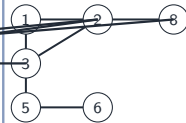
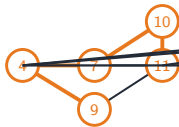
$V(G) \setminus V(T)$

Execução da busca



Busca($G, 7$)

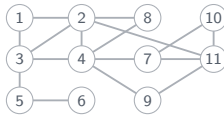
G



$V(T)$

$V(G) \setminus V(T)$

Execução da busca



Busca($G, 7$)

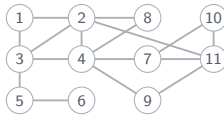
G



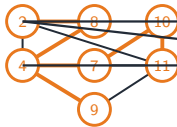
$V(T)$

$V(G) \setminus V(T)$

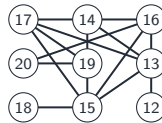
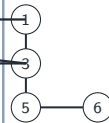
Execução da busca



Busca($G, 7$)



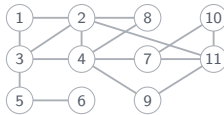
G



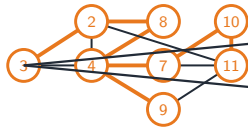
$V(T)$

$V(G) \setminus V(T)$

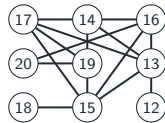
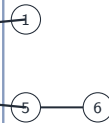
Execução da busca



Busca($G, 7$)



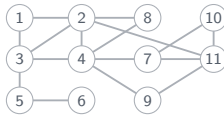
G



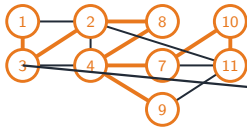
$V(T)$

$V(G) \setminus V(T)$

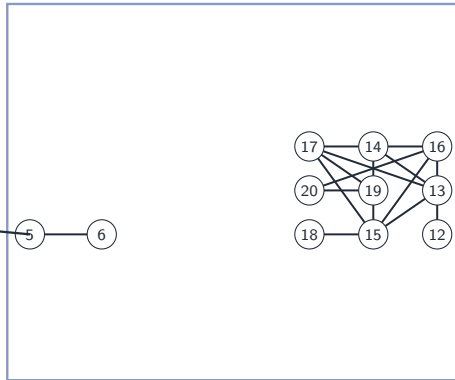
Execução da busca



Busca($G, 7$)



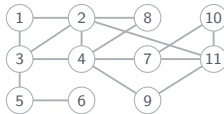
G



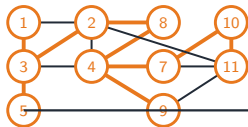
$V(G) \setminus V(T)$

$V(T)$

Execução da busca



Busca($G, 7$)



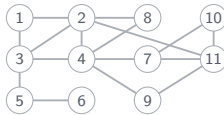
G



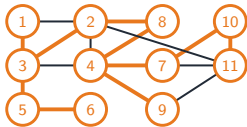
$V(T)$

$V(G) \setminus V(T)$

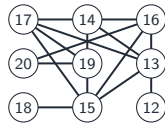
Execução da busca



Busca($G, 7$)



G



$V(T)$

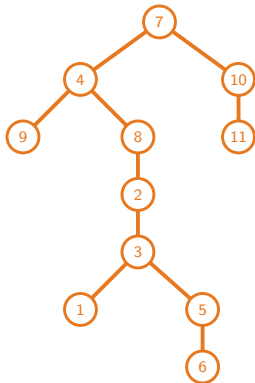
$V(G) \setminus V(T)$



Árvore, caminhos e ciclos

Árvore de busca

- ▶ Todo vértice alcançado tem um predecessor na árvore.



Arestas de T

$$E(T) = \{\{v.\text{pred}, v\} : v \in V(T) \setminus \{s\}\}.$$

- ▶ $v.\text{visitado} = 1$ se, e somente se, v está na árvore.
- ▶ A árvore depende da ordem de escolha das arestas.

Consequências da busca

- ▶ Após a execução de $\text{Busca}(G, s)$:
- ▶ $v.\text{visitado} = 1$ se, e somente se, existe um s - v -caminho;
- ▶ $v.\text{visitado} = 1$ se, e somente se, v e s pertencem à mesma componente conexa;
- ▶ podemos reconstruir um s - v -caminho usando os predecessores:

$$(s, \dots, v.\text{predecessor}, v)$$
$$(s, \dots, v.\text{predecessor}.\text{predecessor}, v.\text{predecessor}, v).$$

Como a busca identifica ciclos?

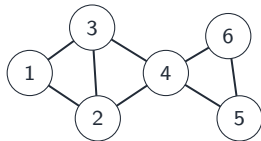
A árvore de busca T não contém ciclos. Entretanto, o grafo original G pode conter ciclos.

Critério em grafo não direcionado

Se durante a busca encontramos uma aresta uv ligando dois vértices já visitados e v não é predecessor de u , então essa aresta fecha um ciclo em G .

Intuição

A árvore já contém um caminho entre u e v . Ao acrescentar a aresta extra uv , obtemos um ciclo.





Busca em profundidade

Busca em profundidade (DFS)

A busca em profundidade explora o grafo seguindo um ramo até onde for possível. Quando não há mais vértices novos a alcançar naquele ramo, o algoritmo retrocede.

Critério de escolha

Dentre todos os vértices marcados e incidentes a alguma aresta ainda não explorada, escolhe-se aquele que foi **mais recentemente alcançado** na busca.

- ▶ tenta avançar o máximo possível em um ramo;
- ▶ ao encontrar um “beco sem saída”, retrocede;
- ▶ gera uma árvore chamada **árvore de profundidade**.

Estrutura usada
Pilha

Último a entrar, primeiro a sair.

BuscaProfundidade(G, s)

- 1 $s.\text{visitado} = 1$
- 2 cria pilha vazia P
- 3 $\text{EMPILHA}(P, s)$
- 4 enquanto $P.\text{tamanho} > 0$ faça
- 5 $u = \text{CONSULTA}(P)$
- 6 se existe $uv \in E(G)$ com $v.\text{visitado} = 0$ então
- 7 $v.\text{visitado} = 1$
- 8 $v.\text{predecessor} = u$
- 9 $\text{EMPILHA}(P, v)$
- 10 senão
- 11 $u = \text{DESEMPILHA}(P)$

Execução do DFS

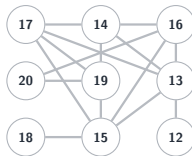
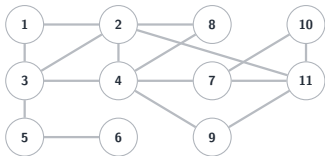


Tabela de controle

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
visitado																				
predecessor																				

Pilha P

topo



base

Resumindo

- ▶ A busca constrói uma árvore T dentro de G .
- ▶ Se $V(T) = V(G)$ ao final, o grafo é conexo; caso contrário, a busca alcança apenas a componente de s .
- ▶ A árvore de busca é acíclica, mesmo que G possua ciclos.
- ▶ Ciclos podem ser detectados por arestas extras entre vértices já visitados.
- ▶ Na DFS, a escolha do próximo vértice segue a lógica de uma pilha.

Exercícios

Usando DFS, escreva um algoritmo para verificar se há ciclo em um grafo direcionado e em um grafo simples.

Leitura e prática

- ▶ Implementar a busca genérica armazenando predecessores.
- ▶ Implementar DFS com lista de adjacência e pilha.
- ▶ Testar a detecção de ciclos em grafos simples.
- ▶ Comparar a árvore obtida ao mudar a ordem dos vizinhos.



Busca em largura

Busca em largura (BFS)

A busca em largura explora o grafo por **níveis**, isto é, primeiro visita os vértices mais próximos do vértice inicial s e, depois, avança para vértices mais distantes.

Critério de escolha

Dentre os vértices alcançados, escolhe-se aquele que foi **alcançado há mais tempo** e que ainda possui vizinhos não visitados.

- ▶ visita primeiro os vizinhos diretos de s ;
- ▶ depois visita os vértices a distância 2 de s ;
- ▶ em seguida, os vértices a distância 3, e assim por diante.

Estrutura usada

Fila

Primeiro a entrar, primeiro a sair.

BuscaLargura(G, s)

- 1 $s.visitado = 1$
- 2 cria fila vazia F
- 3 ENFILEIRA(F, s)
- 4 enquanto $F.tamanho > 0$ faça
- 5 $u = \text{DESENFILEIRA}(F)$
- 6 para todo vértice $v \in N(u)$ faça
- 7 se $v.visitado = 0$ então
- 8 $v.visitado = 1$
- 9 $v.predecessor = u$
- 10 ENFILEIRA(F, v)

Execução do BFS

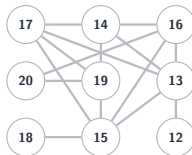
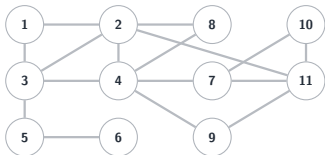
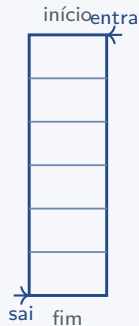


Tabela de controle

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
visitado																				
predecessor																				

Fila F



Leitura e prática

- ▶ Implementar a busca em largura (BFS) armazenando predecessores.
- ▶ Implementar BFS com lista de adjacência e fila.
- ▶ Determinar as distâncias mínimas a partir de um vértice inicial.
- ▶ Verificar os níveis da árvore gerada pela BFS.
- ▶ Comparar a árvore obtida ao mudar a ordem dos vizinhos.

Teoria dos Grafos

Aula 09 – Busca em grafos



Prof. Dr. Robson Medrado de Oliveira

robson@pucgoias.edu.br